

HTML Forms & Java Servlets — GET/POST Assignment Answers

Topics: GET vs POST forms, request.getParameter(), form validation in servlets

Q1. search.html — Song Search Form (GET method)

A simple form asking for a song name and artist, submitting via GET so the values appear as URL query parameters (e.g. SongSearchServlet?song=Blinding+Lights&artist=The+Weeknd).

```
<form action="SongSearchServlet" method="GET">
  <input type="text" name="song" placeholder="Song name" required>
  <input type="text" name="artist" placeholder="Artist name" required>
  <button type="submit">Search</button>
</form>
```

(Full styled file: **search.html**, attached separately.)

Q2. SongSearchServlet — doGet()

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/SongSearchServlet")
public class SongSearchServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String song = request.getParameter("song");
        String artist = request.getParameter("artist");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Searching for " + song + " by " + artist + "</h2>");
        out.println("</body></html>");
    }
}
```

Because the form used `method="GET"`, the servlet reads the values with `request.getParameter()`, the same way regardless of GET or POST — only how the browser transmits the data differs.

Q3. feedback.html — Zomato-style Feedback Form (POST method)

Collects the user's name, order ID, and feedback message, submitted via POST so the data travels in the request body (not visible in the URL).

```
<form action="FeedbackServlet" method="POST">
  <input type="text" name="name" placeholder="Enter your name">
  <input type="text" name="orderId" placeholder="e.g. ZOM12345">
  <textarea name="feedback" rows="4" placeholder="Tell us about your order..."></textarea>
  <button type="submit">Submit Feedback</button>
</form>
```

(Full styled file: **feedback.html**, attached separately.)

Q4. FeedbackServlet — doPost()

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/FeedbackServlet")
public class FeedbackServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("name");
        String orderId = request.getParameter("orderId");
        String feedback = request.getParameter("feedback");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h2>Thank you, " + name + "!</h2>");
        out.println("<p>Order ID: " + orderId + "</p>");
        out.println("<p>Your feedback: " + feedback + "</p>");
        out.println("</body></html>");
    }
}
```

Q5. FeedbackServlet — with Empty-Field Validation

The doPost() method is extended to check whether any field is empty (null or blank after trimming) before showing the thank-you message.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String name = request.getParameter("name");
    String orderId = request.getParameter("orderId");
    String feedback = request.getParameter("feedback");

    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<html><body>");

    if (name == null || name.trim().isEmpty() ||
        orderId == null || orderId.trim().isEmpty() ||
        feedback == null || feedback.trim().isEmpty()) {

        out.println("<h2 style='color:red;'>Error: All fields are required. Please fill in name, order ID, and f
    } else {
        out.println("<h2>Thank you, " + name + "!</h2>");
        out.println("<p>Order ID: " + orderId + "</p>");
        out.println("<p>Your feedback: " + feedback + "</p>");
    }

    out.println("</body></html>");
}
```

Logic: if-else checks each parameter for null or an empty/blank string. If any field fails the check, an error message is shown instead of the thank-you message; otherwise the submitted details are displayed as confirmation.